

C++ ile Nesne Yönelimli File Carving Aracı: İmza Tabanlı Eğitsel Bir Güvenlik Uygulaması

Berke Taha Özdemir

ORCID: 0009-0008-3670-0152

244410043

Bilgisayar Mühendisliği Bölümü

Kastamonu Üniversitesi

Özet - Bu çalışma, C++17 ile geliştirilen nesne yönelimli ve komut satırı tabanlı basit bir file carving uygulamasını sunmaktadır. Uygulama, kullanıcının verdiği yerel ikili dosya içinde JPEG, PNG ve PDF dosya imzalarını arar; başlangıç ve bitiş desenleri eşleştğinde bulunan parçayı ayrı çıktı dosyası olarak kaydeder. Proje, zararlı kullanım veya yetkisiz erişim amacı taşımaz; file carving, güvenlik araçları, ham byte analizi ve nesne yönelimli tasarım prensiplerini eğitsel bir bağlamda göstermek için hazırlanmıştır. Tasarımda sınıflar, kapsülleme, soyutlama, kalıtım ve çok biçimlilik açık şekilde kullanılmıştır. Testlerde sahte PDF parçası ve PNG imzası taşıyan gömülü görsel içeren örnek dosyalar başarıyla tespit edilmiştir.

Anahtar Kelimeler - file carving, C++17, OOP, dijital adli bilişim, güvenlik araçları, dosya imzası, binary analiz

I. GİRİŞ

Dijital adli bilişim ve olay müdahalesi süreçlerinde ham verinin incelenmesi, dosya sistemi bilgilerinin eksik, bozuk veya tamamlanmamış olduğu durumlarda önemli bir rol oynar. File carving yaklaşımı, dosya adlarına veya izin kayıtlarına güvenmek yerine dosya içeriğinde bulunan karakteristik imzalara dayanır. Bu çalışma, söz konusu yaklaşımın temel fikrini öğrenci düzeyinde uygulanabilir bir C++ projesine dönüştürmektedir. Güncel siber güvenlik rehberleri de olay müdahalesinin kurumsal risk yönetimi ve kurtarma faaliyetleriyle ilişkilendirilmesini vurguladığından, NIST CSF 2.0 ve NIST SP 800-61 Rev. 3 bu raporda temel çağdaş güvenlik süreci kaynakları olarak kullanılmıştır [1], [2].

Projenin amacı profesyonel disk kurtarma yazılımı geliştirmek değildir. Bunun yerine, güvenli ve kontrollü bir yerel test dosyası üzerinde JPG, PNG ve PDF imzalarını arayan, bulunan parçaları ayrı dosyalara yazan ve bu süreçte nesne yönelimli programlama prensiplerini açıkça gösteren bir demo araç geliştirilmiştir. Güncel dijital adli bilişim süreç çalışmaları da dijital delil yönetiminde izlenebilirlik, belgelendirme kalitesi ve kanıt işleme disiplininin önemini vurgulamaktadır [3]. Bu nedenle uygulama etik ve savunmacı sınırlar içinde tutulmuştur.

II. KATKI

Bu çalışmanın temel katkısı, file carving kavramını sınıf tabanlı, izlenebilir ve öğrenci düzeyinde anlaşılabilir bir C++17 uygulaması üzerinden somutlaştırmasıdır. Uygulama yalnızca çalışan bir komut satırı aracı olarak değil, aynı zamanda nesne yönelimli programlama ilkelerinin güvenlik aracı geliştirme bağlamında nasıl

kullanılabileceğini gösteren küçük ölçekli bir eğitim örneği olarak tasarlanmıştır.

Çalışma ayrıca imza tabanlı içerik analizini ve dosya uzantılarından bağımsız karar verebilme yeteneğini küçük ve kontrollü test senaryoları üzerinden göstermektedir. Kapsamın JPEG, PNG ve PDF ile sınırlandırılması, projenin etik, açıklanabilir ve ödev hedeflerine uygun kalmasını sağlamıştır.

III. KURAMSAL ARKA PLAN

A. File Carving Nedir?

File carving, ham veri içinde belirli dosya türlerine ait başlangıç ve bitiş desenleri ya da diğer tanımlayıcı yapıların aranarak dosya parçalarının çıkarılmasıdır. Güncel çalışmalar, file carving yaklaşımını dosya sistemi metadata bilgilerinin bozuk, silinmiş veya kullanılamaz olduğu durumlarda başvurulacak içerik tabanlı bir kurtarma tekniği olarak açıklamaktadır [4], [5]. Bu durum, yöntemi eğitsel bir güvenlik aracı senaryosunda düşük seviyeli byte analizini göstermek için uygun hale getirir. PhotoRec gibi araçlar da dosya sistemini yok sayarak alttaki veriye odaklanır; bu nedenle bozulmuş ya da yeniden biçimlendirilmiş ortamlarda faydalı olabilir [9], [10].

Bu projede kullanılan model, footer desenleriyle kontrol edilen basit bir imza tabanlı carving yaklaşımıdır. JPEG için başlangıç deseninde FF D8 FF, bitiş deseninde FF D9; PNG için standart sekiz byte uzunluğundaki PNG imzası ve IEND bloğu; PDF için ise %PDF- başlangıç işareti ve %%EOF bitiş işareti kullanılmıştır. PNG imzası W3C PNG Third Edition standardında tanımlanmaktadır [14]. JPEG/JFIF imza açıklaması Library of Congress format tanımıyla desteklenmiştir [15]. PDF tarafında ise PDF Association tarafından yayımlanan güncel PDF 2.0 errata ve standart bakım notları dikkate alınmıştır [16].

B. Security Tools Kavramı

Security tools, sistemleri korumak ve analiz etmek, olayları incelemek ve güvenlik farkındalığını artırmak için kullanılan yazılım araçlarıdır. Bu kapsamda file carving araçları, veri kurtarma ve adli inceleme süreçlerinin yardımcı bileşenleri olarak değerlendirilebilir. Güncel literatür; PhotoRec, Foremost, Autopsy ve benzeri araçları file carving veya adli kurtarma ortamlarına örnek olarak göstermeye devam etmektedir [4]. Autopsy açık kaynaklı bir dijital adli bilişim platformu olarak sürdürülürken, PhotoRec TestDisk veri kurtarma araç setinin bir parçası olarak güncel biçimde kullanılmaktadır [8], [9].

Benzer şekilde bulk_extractor, dosya sistemi yapısını ayrıştırmadan disk imajları, dosyalar veya izinler üzerinde

yapılandırılmış adli bilgileri arayabilir [11]. YARA, metinsel ve ikili desenler üzerinden kural tabanlı tanımlamayı destekler; libmagic ise magic-number tabanlı dosya türü tanımının başka bir örneğini sunar [12], [13]. Bu proje, söz konusu profesyonel araçların sınırlı ve eğitsel bir minyatürü olarak düşünülmelidir.

IV. PROJE AMACI VE KAPSAMI

Geliştirilen uygulama, komut satırından iki argüman alır: giriş dosyası yolu ve çıktı klasörü yolu. Program yalnızca kullanıcının sağladığı yerel test dosyaları üzerinde çalışır. Yetkisiz cihaz, disk, ağ kaynağı veya üçüncü kişilere ait veri üzerinde işlem yapmak hedeflenmemiştir. Bu karar; sınırlı yetki, kontrollü giriş ve açık yazılım sınırlarının vurgulandığı güncel güvenli tasarım yaklaşımıyla uyumludur [20].

Kapsam özellikle dar tutulmuştur: araç yalnızca JPEG, PNG ve PDF imzalarını tarar; bozuk dosyaları onarmaz, parçalanmış dosyaları birleştirmez, NTFS/FAT gibi dosya sistemlerini analiz etmez ve şifreli içerikleri çözmeye çalışmaz. Bu sınırlama, ödevin OOP hedefini netleştirmek ve çalışan, açıklanabilir bir eğitim demosu üretmek için bilinçli olarak seçilmiştir.

V. SINIF TASARIMI VE OOP PRENSİPLERİ

Tasarımda her sınıfa tek ve anlaşılır bir sorumluluk verilmiştir. BinaryFileReader dosya okuma, PatternUtils byte desen arama, FileSignature dosya imzası soyutlaması, FileCarver tarama motoru ve OutputWriter çıktı yazma görevlerini üstlenir. Böylece main fonksiyonu sade tutulmuş, program akışı CommandLineApplication sınıfında koordine edilmiştir.

TABLE I. PROJEDE KULLANILAN SINIFLAR VE SORUMLULUKLARI

BinaryFileReader: Giriş dosyasını binary modda okuyup byte vektörü döndürür.
PatternUtils: Byte dizisi içinde desen eşleşmesi ve ilk eşleşme araması yapar.
FileSignature: Dosya türü adı, uzantı ve header bilgisini soyutlar.
FixedFooterSignature: Header sonrasında footer arayan ortak kalıtım sınıfıdır.
Jpeg/Png/PdfSignature: Desteklenen dosya türlerinin imza bilgilerini tanımlar.
FileCarver: Kayıtlı imzalarla veriyi tarar ve fragment listesi üretir.
OutputWriter: Bulunan fragmentleri output klasörüne dosya olarak yazar.
CommandLineApplication: Komut satırı argümanlarını ve genel iş akışını yönetir.

A. Kapsülleme

Sınıfların iç verileri dışarıdan doğrudan değiştirilemez. Örneğin FileCarver içinde tutulan imza listesi private alandadır ve yalnızca addSignature fonksiyonu ile genişletilir. Bu yaklaşım, sınıf iç durumunun kontrollü biçimde yönetilmesini sağlar.

B. Soyutlama, Kalıtım ve Çok Biçimlilik

FileSignature soyut temel sınıfı, farklı dosya türlerinin ortak davranışlarını tanımlar. FixedFooterSignature bu sınıftan türeyerek footer arama mantığını ortaklaştırır. JpegSignature, PngSignature ve PdfSignature sınıfları ise aynı tabandan türeyen somut imza sınıflarıdır. FileCarver bu sınıfları std::unique_ptr<FileSignature> vektörü içinde tutar; böylece tarama motoru somut türü bilmeden ortak arayüz üzerinden çalışır. Bu yapı, C++ Core Guidelines tarafından önerilen sahiplik ve kaynak yönetimi anlayışıyla da uyumludur [17].

FileSignature

```
-> FixedFooterSignature
-> JpegSignature
-> PngSignature
```

-> PdfSignature

FileCarver

```
-> vector<unique_ptr<FileSignature>>
```

Fig. 1. Sınıf ilişkilerinin özet görünümü.

VI. ÇEKİRDEK ALGORİTMA

Algoritma önce dosyayı ham byte vektörü olarak belleğe alır. Ardından her byte pozisyonunda kayıtlı imzalar sırayla denenir. Başlangıç imzası eşleşirse ilgili dosya türünün footer deseni aranır. Geçerli footer bulunursa başlangıç ve bitiş arasındaki veri CarvedFragment nesnesine kopyalanır. Aynı fragmentin tekrar bulunmasını önlemek için tarama konumu fragment bitimine ilerletilir.

for position in data:

for signature in signatures:

if signature.matchesStart(data, position):

end = signature.findEndExclusive(data, position)

if end is valid:

create CarvedFragment

position = end

Fig. 2. FileCarver tarama algoritmasının yalın sözde kodu.

Bu modelin basit çalışma zamanı, veri uzunluğu ile imza sayısının çarpımına bağlıdır. Desteklenen imza sayısı üç ile sınırlı olduğu için öğrenci düzeyindeki test dosyalarında performans problemi beklenmez. Daha büyük sistemlerde çoklu desen arama algoritmaları, yapılandırılabilir imza veritabanları veya modern dosya parçası sınıflandırma yaklaşımları tercih edilebilir [5]. YARA ve libmagic gibi araçlar da desen tabanlı tanıma fikrinin daha gelişmiş örneklerini sunar [12], [13].

VII. C++17 UYGULAMA KARARLARI

Uygulamada std::vector<unsigned char> ham dosya verisini saklamak için kullanılmıştır. std::optional<std::size_t>, footer bulunamadığında boş sonuç döndürmek için tercih edilmiştir. std::unique_ptr<FileSignature> ise imza nesnelerinin sahipliğini FileCarver içinde açık hale getirir ve polimorfik kullanım sağlar. C++17 filesystem kütüphanesi output klasörünü oluşturmak ve çıktı dosya yollarını platformdan bağımsız biçimde yönetmek için kullanılmıştır [18], [19].

Kodun güvenli tutulması için program yalnızca belirtilen dosya yolunu okur ve yalnızca belirtilen çıktı klasörüne yazar. Giriş dosyası açılmadığında exception yakalanır ve kullanıcıya açık hata mesajı verilir. Bu yaklaşım hem komut satırı araçlarında anlaşılabilir hata yönetimi sağlar hem de ödev kapsamında gereksiz yetki, ağ erişimi veya saldırı yeteneği eklenmesini engeller [20].

VIII. TEST SENARYOLARI VE BULGULAR

Testlerde öncelikle içinde yalnızca metin bulunan test.bin dosyası kullanılmıştır. Bu dosyada desteklenen dosya imzası bulunmadığı için program 0 fragment sonucu üretmiştir. Ardından sahte PDF başlangıç ve bitiş imzası içeren test_pdf.bin dosyası hazırlanmış; program bu dosyadan PDF fragmentini başarıyla çıkarmıştır. Son olarak adı test_jpg.bin olan ancak içinde PNG imzası taşıyan gömülü görsel içeren dosya denenmiştir. Program dosya adına değil imzaya baktığı için doğru şekilde PNG fragmenti üretmiştir.

TABLE II. TEST SENARYOLARI VE BEKLENEN/GERÇEK SONUÇLAR

- Yalnızca metin içeren test.bin | Beklenen: 0 fragment | Gerçek: 0 fragment bulundu.
- Sahte PDF imzalı test_pdf.bin | Beklenen: 1 PDF fragment | Gerçek: PDF başlangıç 18, çıktı .pdf.

3. Gömülü görsel içerden test_jpg.bin | Beklenen: 1 görsel fragment | Gerçek: PNG başlangıç 18, çıktı .png.
4. Eksik footer içerden dosya | Beklenen: Çıktı üretmemeli | Gerçek: Footer yoksa fragment oluşturulmaz.
5. Birden fazla gömülü dosya | Beklenen: Her biri ayrı yazılmalı | Gerçek: Algoritma çoklu fragment destekler.

Elde edilen örnek çıktıda 9407 byte okunan test_jpg.bin dosyasında 1 fragment bulunmuş, fragmentin PNG olduğu tespit edilmiş ve output\fragment_001_PNG_18.png adıyla yazılmıştır. Bu sonuç, araç tasarımının dosya uzantısına değil içerik imzasına göre karar verdiğini göstermektedir.

IX. SINIRLILIKLAR VE ETİK KAPSAM

Bu çalışma profesyonel forensic yazılımların yerini almak için tasarlanmamıştır. Program disk imajı ayrıştırmaz, silinmiş dosyaları dosya sistemi düzeyinde kurtarmaz, parçalanmış dosyaları birleştirmez ve bozuk dosyaları onarmaz. Ayrıca şifreli veya sıkıştırılmış karmaşık yapıların içeriğini çözmeye çalışmaz. Bu sınırlılıklar, projenin güvenli, anlaşılabilir ve OOP odaklı kalmasını sağlar.

Etik kullanım açısından uygulama yalnızca kullanıcının sahip olduğu veya test amacıyla oluşturduğu yerel dosyalar üzerinde çalıştırılmalıdır. Bu yaklaşım, dijital adli bilişim süreçlerinde izlenebilirlik, belgelendirme kalitesi ve prosedürel tutarlılığın önemini vurgulayan güncel araştırmalarla uyumludur [3]. Gelecekte uygulamaya grafiksel arayüz, daha fazla dosya türü, yapılandırılabilir imza listesi ve detaylı loglama eklenebilir; ancak çekirdek motorun komut satırı tabanlı kalması güvenlik araçlarında yaygın olan otomasyon kullanımını kolaylaştırır.

X. SONUÇ

Bu projede C++17 ile nesne yönelimli programlama prensiplerine dayalı basit ve eğitsel bir file carving aracı geliştirilmiştir. Program komut satırından aldığı yerel binary dosyayı okuyarak JPEG, PNG ve PDF dosya imzalarını aramakta, bulunan fragmentleri ayrı çıktı dosyalarına yazmaktadır. Tasarımda sınıflar, kapsüller, soyutlama, kalıtım ve çok biçimlilik açık biçimde uygulanmıştır.

Sonuç olarak uygulama, hem file carving kavramını hem de C++ OOP tasarımını göstermeye uygun bir üniversite ödevi demosu oluşturmuştur. Kodun kapsamı bilinçli olarak basit tutulmuş, güvenlik konusu etik ve savunmacı çerçevede ele alınmıştır. Test çıktıları, imza tabanlı carving mantığının çalıştığını ve programın dosya adından bağımsız olarak gerçek dosya türünü byte imzalarına göre tespit edebildiğini göstermiştir.

XI. GELECEK ÇALIŞMALAR

Gelecekte uygulama; yapılandırılabilir imza listeleri, JPEG/PNG/PDF dışındaki dosya türleri ve tüm veriyi belleğe almak yerine büyük dosyaları parça parça okuyabilen daha ölçeklenebilir bir mimari ile genişletilebilir. Ayrıca çıkarılan fragmentler için hash üretimi, zaman damgalı loglama ve basit raporlama çıktıları eklenerek adli inceleme süreçlerine daha yakın bir eğitim senaryosu oluşturulabilir [3], [7].

Sonraki aşamada false positive sonuçları azaltmak için YARA benzeri desen tanımları, libmagic tabanlı doğrulama mekanizmaları veya dosya fragmentleri için daha gelişmiş sınıflandırma teknikleri kullanılabilir [5], [12], [13]. Bu tür eklemeler, mevcut çekirdek carving motorunu korurken aracı gelecekte daha sürdürülebilir, genişletilebilir ve laboratuvar kullanımına uygun bir güvenlik eğitimi platformuna dönüştürülebilir.

KAYNAKLAR

- [1] NIST, "The NIST Cybersecurity Framework (CSF) 2.0," NIST Cybersecurity White Paper 29, 26 Şubat 2024. [Çevrimiçi]. Erişim: <https://csrc.nist.gov/pubs/cswp/29/the-nist-cybersecurity-framework-csf-20/final>. Erişim tarihi: 25 Mayıs 2026.

- [2] A. Nelson, S. Rekhi, M. Souppaya ve K. Scarfone, "Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile," NIST SP 800-61 Rev. 3, Nisan 2025. [Çevrimiçi]. Erişim: <https://doi.org/10.6028/NIST.SP.800-61r3>. Erişim tarihi: 25 Mayıs 2026.
- [3] Z. Moric, V. Dakić ve I. Ogrizek Biškupić, "An Empirical Assessment of Digital Forensic Process Reliability Using Integrated ISO/IEC 27037 and 27041 Standards," Journal of Cybersecurity and Privacy, c. 6, sy. 2, makale 57, 30 Mart 2026. [Çevrimiçi]. Erişim: <https://doi.org/10.3390/jcp6020057>. Erişim tarihi: 25 Mayıs 2026.
- [4] P. C. Sethi, "File Carving: Analyzing Data Retrieval in Digital Forensics," International Journal of Computer and Information Technology, c. 13, sy. 3, Eylül 2024. [Çevrimiçi]. Erişim: <https://ijcit.com/index.php/ijcit/article/view/420/109>. Erişim tarihi: 25 Mayıs 2026.
- [5] A. Guzhov ve C. T. Wirth, "Transformer-Based File Fragment Type Classification for File Carving in Digital Forensics," 24th European Conference on Cyber Warfare and Security (ECCWS 2025) Bildirileri, 2025. [Çevrimiçi]. Erişim: https://www.dfki.de/fileadmin/user_upload/import/16030_Guzhov_ECCWS_2025.pdf. Erişim tarihi: 25 Mayıs 2026.
- [6] K. Hughes ve M. Black, "An Alternative Approach to Data Carving PDF Files," Journal of Cybersecurity Education, Research and Practice, c. 2024, sy. 1, makale 21, 2024. [Çevrimiçi]. Erişim: <https://digitalcommons.kennesaw.edu/jcerp/vol2024/iss1/21/>. Erişim tarihi: 25 Mayıs 2026.
- [7] M. Stanković ve T. M. Khan, "Digital Forensics Tool Evaluation on Deleted Files," Digital Forensics and Cyber Crime içinde, Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, Springer, 2023, ss. 61-83. [Çevrimiçi]. Erişim: https://doi.org/10.1007/978-3-031-36574-4_4. Erişim tarihi: 25 Mayıs 2026.
- [8] The Sleuth Kit, "Autopsy User Documentation: Autopsy User's Guide 4.22.0," 2025. [Çevrimiçi]. Erişim: <https://sleuthkit.org/autopsy/docs/user-docs/4.22.0/>. Erişim tarihi: 25 Mayıs 2026.
- [9] CGSecurity, "TestDisk Download: TestDisk & PhotoRec 7.2," 22 Şubat 2024. [Çevrimiçi]. Erişim: https://www.cgsecurity.org/wiki/TestDisk_Download. Erişim tarihi: 25 Mayıs 2026.
- [10] CGSecurity, "PhotoRec, Digital Picture and File Recovery," 21 Mayıs 2023. [Çevrimiçi]. Erişim: <https://www.cgsecurity.org/wiki/PhotoRec>. Erişim tarihi: 25 Mayıs 2026.
- [11] S. Garfinkel, "bulk_extractor," GitHub repository, sürüm bilgileri 2024 içinde güncellenmiştir. [Çevrimiçi]. Erişim: https://github.com/simsong/bulk_extractor. Erişim tarihi: 25 Mayıs 2026.
- [12] VirusTotal, "YARA Releases," GitHub, YARA v4.5.5, 30 Ekim 2025. [Çevrimiçi]. Erişim: <https://github.com/VirusTotal/yara/releases>. Erişim tarihi: 25 Mayıs 2026.
- [13] M. Kerisk, "libmagic(3) - Linux manual page," man7.org, 17 Mayıs 2026 tarihli upstream commit üzerinden oluşturulmuş sayfa. [Çevrimiçi]. Erişim: <https://man7.org/linux/man-pages/man3/libmagic.3.html>. Erişim tarihi: 25 Mayıs 2026.
- [14] W3C, "Portable Network Graphics (PNG) Specification (Third Edition)," W3C Recommendation, 24 Haziran 2025. [Çevrimiçi]. Erişim: <https://www.w3.org/TR/png-3/>. Erişim tarihi: 25 Mayıs 2026.
- [15] Library of Congress, "JPEG File Interchange Format Family," Digital Formats, son güncelleme: 8 Mayıs 2024. [Çevrimiçi]. Erişim: <https://www.loc.gov/preservation/digital/formats/fdd/fdd000619.shtml>. Erişim tarihi: 25 Mayıs 2026.
- [16] PDF Association, "PDF 2.0 Errata Collection 2 is now available," 25 Temmuz 2024. [Çevrimiçi]. Erişim: <https://pdfa.org/pdf-2-0-errata-collection-2-is-now-available/>. Erişim tarihi: 25 Mayıs 2026.
- [17] B. Stroustrup ve H. Sutter, "C++ Core Guidelines," 8 Temmuz 2025. [Çevrimiçi]. Erişim: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>. Erişim tarihi: 25 Mayıs 2026.
- [18] Microsoft, "<filesystem> functions," Microsoft Learn, 23 Nisan 2025. [Çevrimiçi]. Erişim: <https://learn.microsoft.com/en-us/cpp/standard-library/filesystem-functions>. Erişim tarihi: 25 Mayıs 2026.
- [19] Microsoft, "How to: Create and use unique_ptr instances," Microsoft Learn, 13 Nisan 2026. [Çevrimiçi]. Erişim: <https://learn.microsoft.com/en-us/cpp/cpp/how-to-create-and-use-unique-ptr-instances>. Erişim tarihi: 25 Mayıs 2026.
- [20] CISA, NSA, FBI ve uluslararası ortaklar, "Shifting the Balance of Cybersecurity Risk: Principles and Approaches for Secure by Design Software," Ekim 2023. [Çevrimiçi]. Erişim: https://www.cisa.gov/sites/default/files/2023-10/SecureByDesign_1025_508c.pdf. Erişim tarihi: 25 Mayıs 2026.